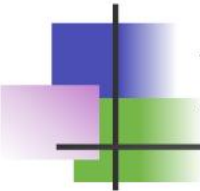




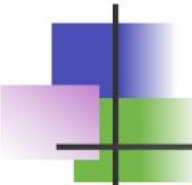
第五章 系统测试

主要内容：系统测试至少占据了制作过程的一半工作量，是成功地开发信息系统的重要保证之一。本章将对系统测试的方法和技术进行系统地介绍。



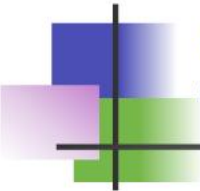
第五章 系统测试

- 5.1 系统测试概述
- 5.2 人工测试
- 5.3 测试数据的设计
- 5.4 模块测试
- 5.5 高级测试
- 5.6 测试计划和控制
- 5.7 程序复杂度的定量度量（新增）
- 思考题



5.7 程序复杂程度的定量度量

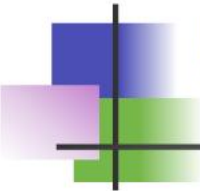
- 1、引言
- 详细设计阶段设计出的模块质量如何？
 - 定性的衡量方法：
 - 依据软件设计的基本原理和概念可仔细衡量。
 - 定量的度量方法：
 - 目前许多定量度量方法还处于研究过程中。
- 本节将要介绍的程序复杂程度定量度量方法是其中比较成熟的一种。



2、重要性

- 估算软件中错误的数量、软件开发所需的工作量
 - 把程序的复杂程度乘以适当常数即可估算出；
- 比较两个不同的设计或两个不同算法的优劣
 - 依定量度量的结果；
- 模块规模的精确限度
 - 依程序的定量的复杂程度

下面着重介绍McCabe方法和Halstead方法。



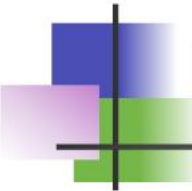
5.7.1 McCabe方法

1. 流图

- McCabe方法（环形复杂度）：
 - 根据程序控制流的复杂程度定量度量程序的复杂程度，这样度量的结果称为程序的环形复杂度。
- 流图(程序图)：
 - 所谓流图实质上是“退化了的”程序流程图，
 - 它仅仅描绘程序的控制流程，完全不表现对数据的具体操作以及分支或循环的具体条件。

程序流程图退化为流图的方法

- 结点：圆表示。代表一条或多条语句。 ○
 - 程序流程图中的一个顺序的处理框序列和一个菱形判定框，可以映射成流图中的一个结点。
- 边：箭头线。代表控制流。 →
 - 流图中一条边必须终止于一个结点，即使这个结点并不代表任何语句(实际上相当于一个空语句)。
- 判定节点：包含条件的结点。
 - 从每个判定结点引出两条或多条边。
- 区域：由边和结点围成的面积称为区域。
 - 计算区域数时应包括图外部未被围起来的那个区域。
- 复杂度：区域数。



复合条件的映射

- 复合条件：

- 即条件中包含了一个或多个布尔运算符：
 - 逻辑OR, AND, NAND, NOR。

- 映射：

- 把复合条件分解为若干个简单条件，
- 每个简单条件对应流图中一个判定结点。

例1、把流程图映射成流图

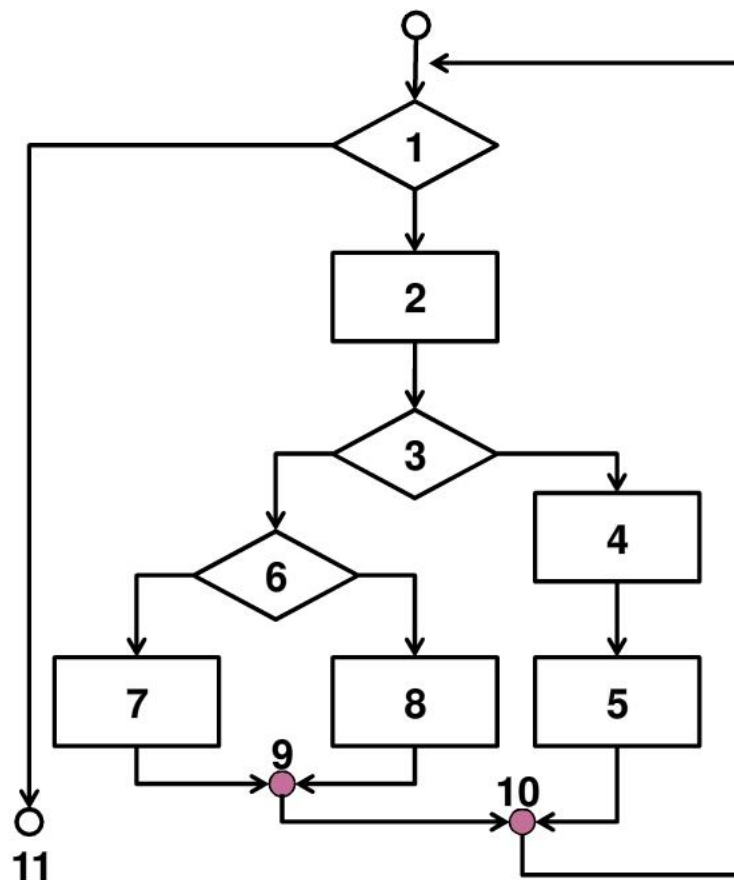


图5.7-1 (a) 程序流程图

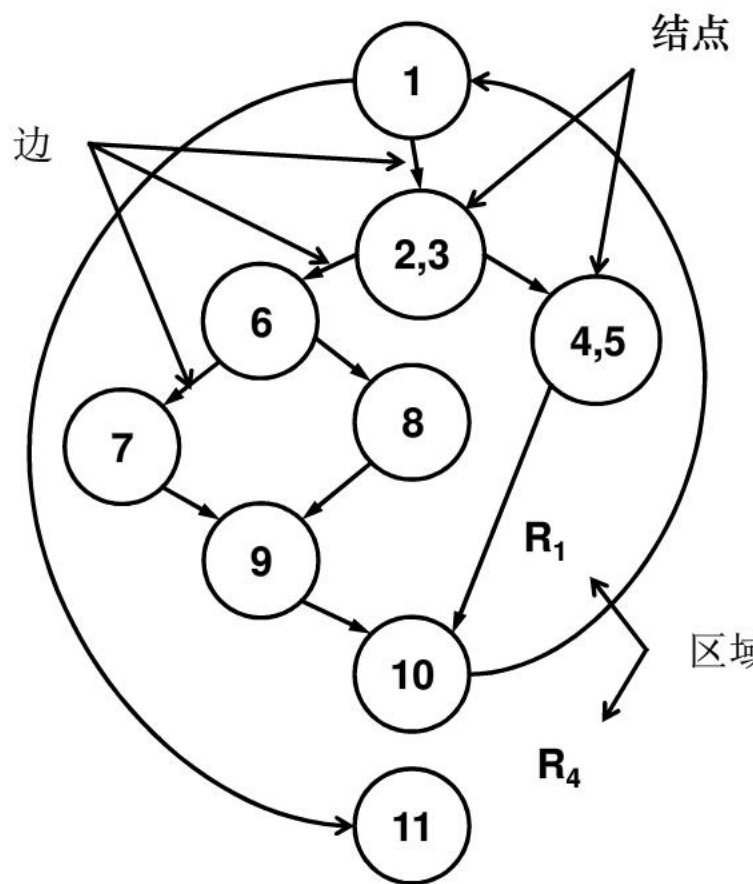


图5.7-2 (b) 流图

例2、由PDL翻译成流图

PDL

Procedure: sort

```
1:  do while records remain
2:    read record;
    if record field1=0;
3:      then process record;
        store in buffer;
        increment counter;
4:    elseif record field2=0;
5:      then reset counter;
6:    else process record;
        store in file;
7a:    endif
    endif
7b: enddo
8:  end
```

流图

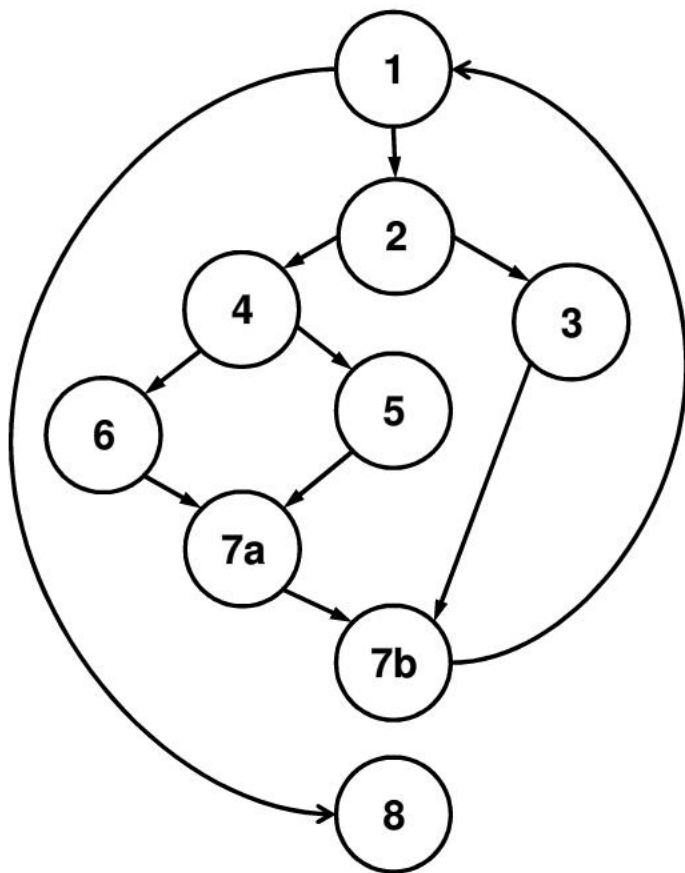


图5 由PDL翻译成的流图

例3、 包含复合条件的PDL映射成流图

```
·  
·  
·  
IF a OR b  
    THEN procedure x  
    ELSE procedure y  
ENDIF  
·  
·  
·
```

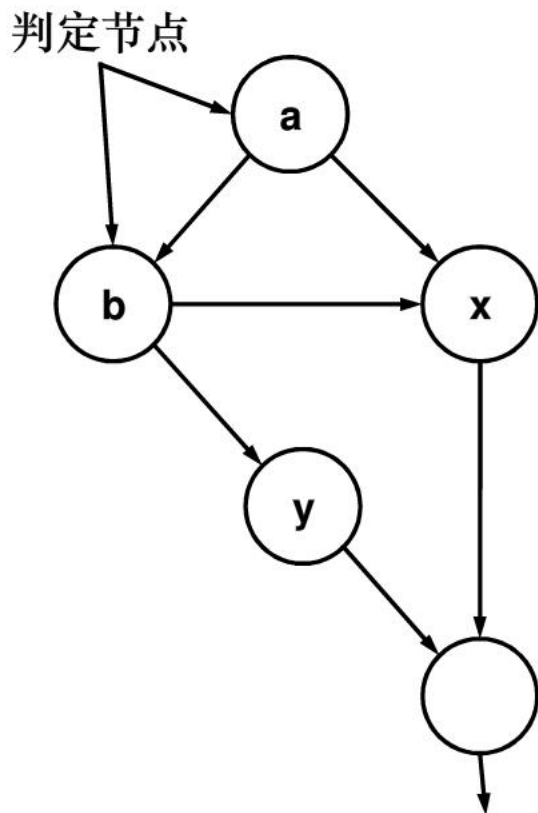
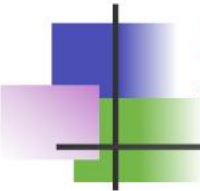


图5.2 由包含复合条件的PDL映射成的流图



2. 计算环形复杂度的方法

- 环形复杂度：定量度量程序的逻辑复杂度。
- 计算环形复杂度：基于程序控制流的流图。
- 3种方法（计算环形复杂度）：
 - (1) 流图中的区域数等于环形复杂度。
 - (2) 流图G的环形复杂度 $V(G)=E-N+2$ ，其中，E是流图中边的条数，N是结点数。
 - (3) 流图G的环形复杂度 $V(G)=P+1$ ，其中，P是流图中判定结点的数目。



3. 环形复杂度的用途

■影响程序的环形复杂度的因素：

- 程序控制流的复杂程度，即程序结构的复杂程度。
- 程序内分支数或循环个数。当它们增加时，环形复杂度也随之增加。

■环形复杂度的用途

- 是对测试难度的一种定量度量；
- 能对软件最终的可靠性给出某种预测。

■环形复杂度的值域

- McCabe研究大量程序后发现，环形复杂度高的程序往往是最困难、最容易出问题的程序。
- 实践表明，模块规模以 $V(G) \leq 10$ 为宜， $V(G)=10$ 是模块规模的一个更科学更精确的上限。



5.7.2 Halstead方法

- Halstead方法：

- 根据程序中：

- 运算符+操作数

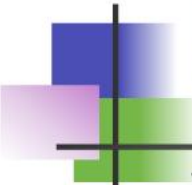
的总数来度量程序的复杂程度

- 令：

- N_1 为程序中运算符出现的总次数；

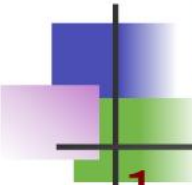
- N_2 为操作数出现的总次数；

- 程序长度 N 定义为： $N=N_1+N_2$



5.7.2 Halstead方法 (续)

- 详细设计完成之后，可知道程序中使用的：
 - n_1 ：不同运算符(包括关键字)的个数；
 - n_2 ：不同操作数(变量和常数)的个数。
- Halstead的预测程序长度公式：
 - $H = n_1 \log_2 n_1 + n_2 \log_2 n_2$
- 验证表明：
 - 预测的长度H与实际长度N非常接近。
- Halstead的预测程序中包含错误个数的公式：
 - $E = N \log_2 (n_1 + n_2) / 3000 \approx H \log_2 (n_1 + n_2) / 3000$
- 对300条到12000条语句范围内的程序核实上述公式，发现预测的错误数与实际错误数相比**误差 $\leq 8\%$** 。



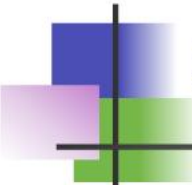
5.7.3 控制结构测试

1. 基本路径测试

- Tom McCabe提出的一种白盒测试技术。
- 用**基本路径测试**设计测试用例的方法：
 - 首先计算程序的环形复杂度；
 - 用该复杂度为指南定义执行路径的基本集合；
 - 从该基本集合导出的测试用例可以保证程序中的每条语句至少执行一次，
 - 而且每个条件在执行时都将分别取真、假两种值。

用基本路径测试技术设计测试用例的步骤

- 第一步，根据过程设计结果画出相应的流图。
- 第二步，计算流图的环形复杂度。
- 第三步，确定线性独立路径的基本集合。
- 第四步，设计可强制执行基本集合中每条路径的测试用例。



举例

- 用基本路径测试技术测试下列的用PDL描述的求平均值过程。
- 注意：为了正确地画出流图，这里将被映射为流图结点的PDL语句编了序号。

PROCEDURE average;

/* 这个过程计算不超过100个在规定值域内的有效数字的平均值
；同时计算有效数字的总和及个数。*/

INTERFACE RETURNS average, total.input, total.valid;

INTERFACE ACCEPTS value, minimum, maximum;

TYPE value [1...100] IS SCALAR ARRAY;

TYPE average, total.input, total.valid;

minimum, maximum, sum IS SCALAR;

TYPE i IS INTEGER;

举例（程序主体部分-见黑板）

1: i=1; total.input=total.valid=0;
sum=0;

2: DO WHILE value[i]<> -999

3: AND total.input<100

4: increment total.input by 1;

5: IF value[i]>=minimum

6: AND value[i]<=maximum

7: THEN increment total.valid by 1;

 sum=sum+value[i];

8: ENDIF

 increment i by 1;

9: ENDDO

10: IF total.valid>0

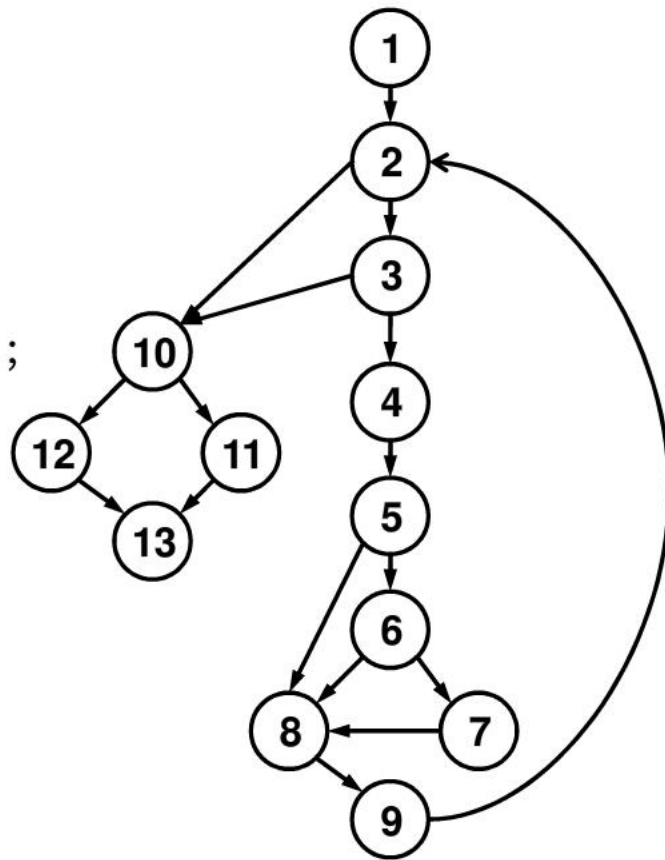
11: THEN average=sum/total.valid;

12: ELSE average=-999;

13: ENDIF

END average

(1) 流图如下。



(2) 计算流图的环形复杂度

3种方法均可

■ 边: $E=17$

■ 结点: $N=13$

■ 判定结点数: $P=5$

■ 环形复杂度:

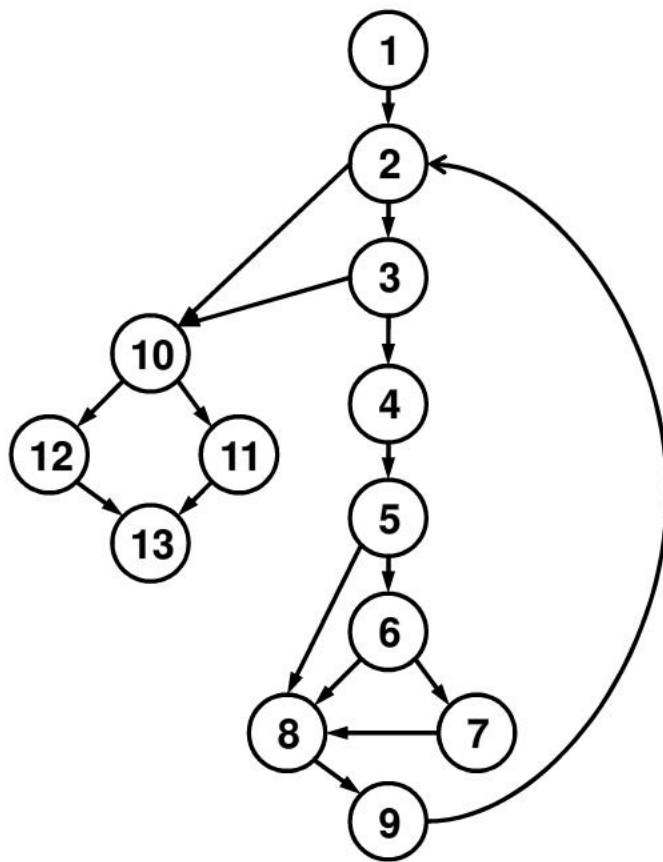
■ $V(G)=E-N+2=6$

■ $V(G)=P+1=6$

■ 区域: 6

■ 判定结点:

■ 2、3、5、6和10。





(3) 确定线性独立路径的基本集合

■ 独立路径：

- 指至少引入程序的一个新处理语句集合或一个新条件的路径；
- 用流图术语描述：独立路径至少包含一条在定义该路径之前不曾用过的边。

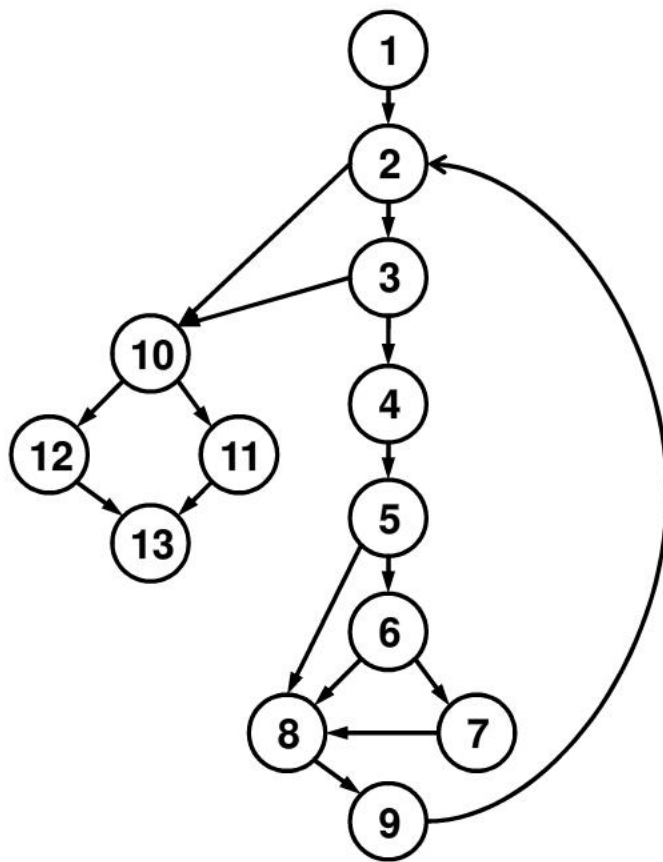
■ 独立路径数：即环形复杂度。

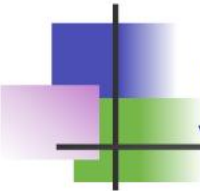
- 环形复杂度是确保程序中所有语句至少被执行一次所需的测试数量的上界。

■ 本例中求平均值过程，由于环形复杂度为6，因此共有**6条独立路径**。

6条独立路径

- 路径1: 1-2-10-11-13
- 路径2: 1-2-10-12-13
- 路径3: 1-2-3-10-11-13
- 路径4: 1-2-3-4-5-8-9-2-...
- 路径5: 1-2-3-4-5-6-8-9-2-...
- 路径6: 1-2-3-4-5-6-7-8-9-2-...





(4) 设计可强制执行基本集合中每条路径的测试用例

■ 设计规则：

- 应该选取测试数据使得在测试每条路径时都适当地设置好了各个判定结点的条件。

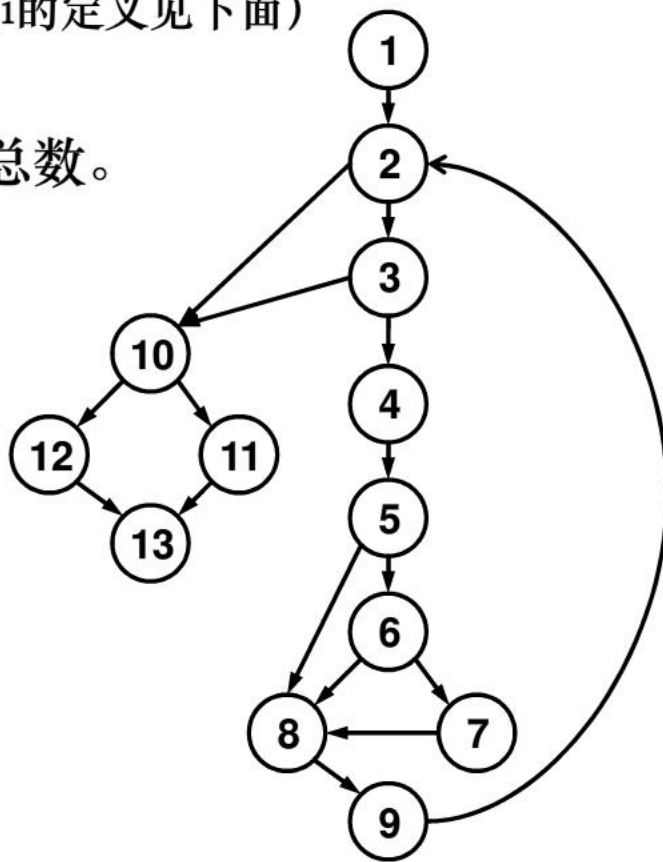
■ 注意：

- 在测试过程中，执行每个测试用例并把实际输出结果与预期结果相比较。
- 一旦执行完所有测试用例，就可以确保程序中所有语句都至少被执行了一次，而且每个条件都分别取过true值和false值。

路径1的测试用例：

- 路径1：1-2-10-11-13
- $value[k]$ =有效输入值，其中 $k < i$ ，（i的定义见下面）
- $value[i] = -999$ ， 其中 $2 \leq i \leq 100$
- 预期结果：基于k的正确平均值和总数。
- 注意：路径1无法独立测试，必须作为路径4、5和6的一部分来测试。

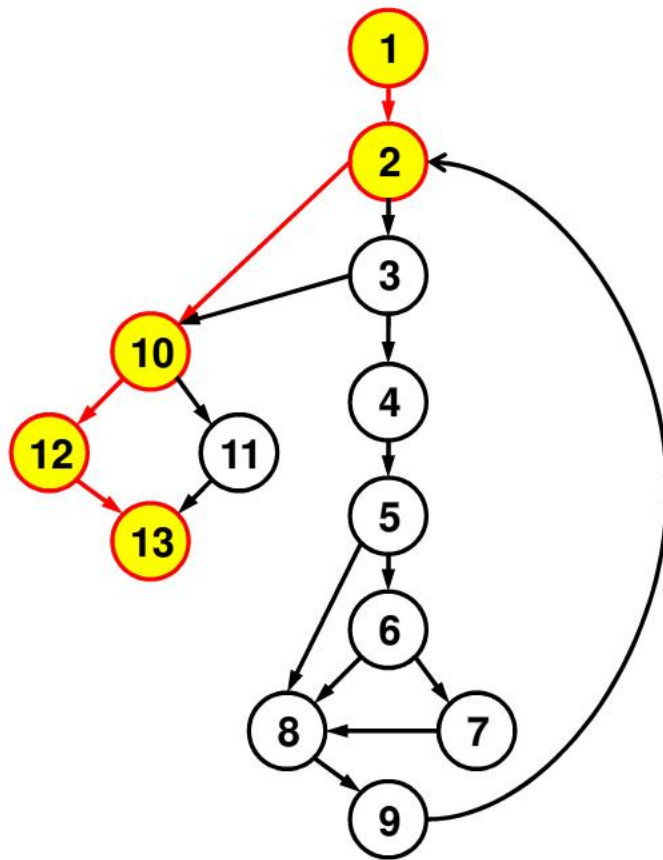
```
1:  i=1; total.input=total.valid=0; sum=0;
2:  DO WHILE  $value[i] \neq -999$       F
3:      AND total.input < 100
...
9:  ENDDO
10: IF  $total.valid > 0$       T
11: THEN average=sum/total.valid;
12: ELSE average=-999;
13: ENDIF
    END average
```



路径2的测试用例:

- 路径2: 1-2-10-12-13
- $\text{value}[1] = -999$
- 预期结果: $\text{average} = -999$,
 - 其他都保持初始值。

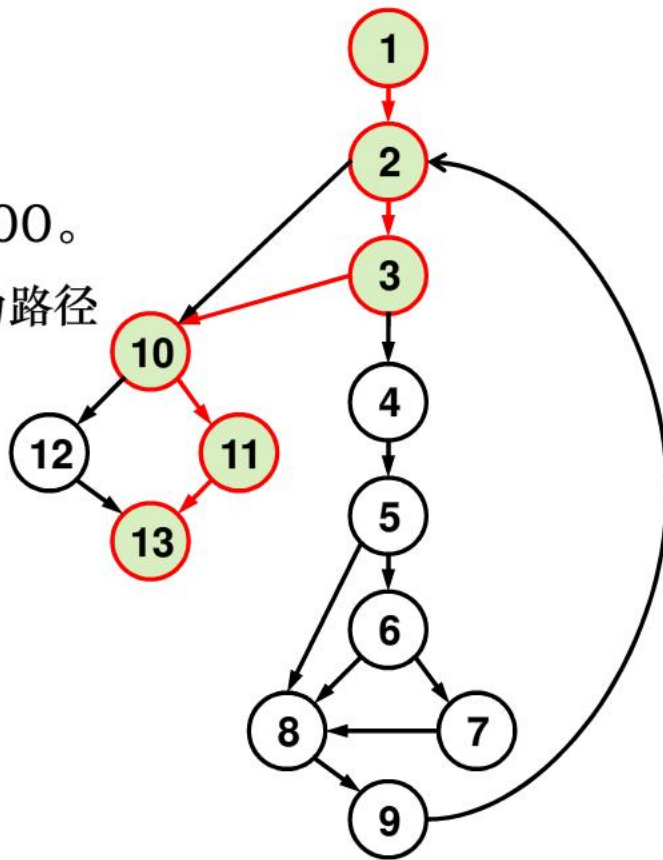
```
1:  i=1; total.input=total.valid=0; sum=0;
2:  DO WHILE value[i]<>-999           F
3:      AND total.input<100
...
9:  ENDDO
10: IF total.valid>0
11: THEN average=sum/total.valid;
12: ELSE average=-999;
13: ENDIF
    END average
```



路径3的测试用例：

- 路径3： 1-2-3-10-11-13
- 试图处理101个或更多个值
- 前100个数值应是有效输入值
- 预期结果：
 - 前100个数的平均值，总数为100。
- 注意： 路径3也无法独立测试，必须作为路径4、5和6的一部分来测试。

```
1:  i=1; total.input=total.valid=0; sum=0;
2:  DO WHILE value[i]<> -999
3:      AND total.input<100      T
...
9:  ENDDO
10: IF total.valid>0
11: THEN average=sum/total.valid;
12: ELSE average=-999;
13: ENDIF
    END average
```



路径4的测试用例：

- 路径4：1-2-3-4-5-8-9-2-...
- $value[i]$ = 有效输入值，其中 $i < 100$ ，
- $value[k] < minimum$ ，其中 $k < i$
- 预期结果：基于k的正确平均值和总数。

1: $i=1$; $total.input=total.valid=0$; $sum=0$;

2: DO WHILE $value[i] < -999$ T

3: AND $total.input < 100$ T

4: increment $total.input$ by 1;

5: IF $value[i] \geq minimum$ F

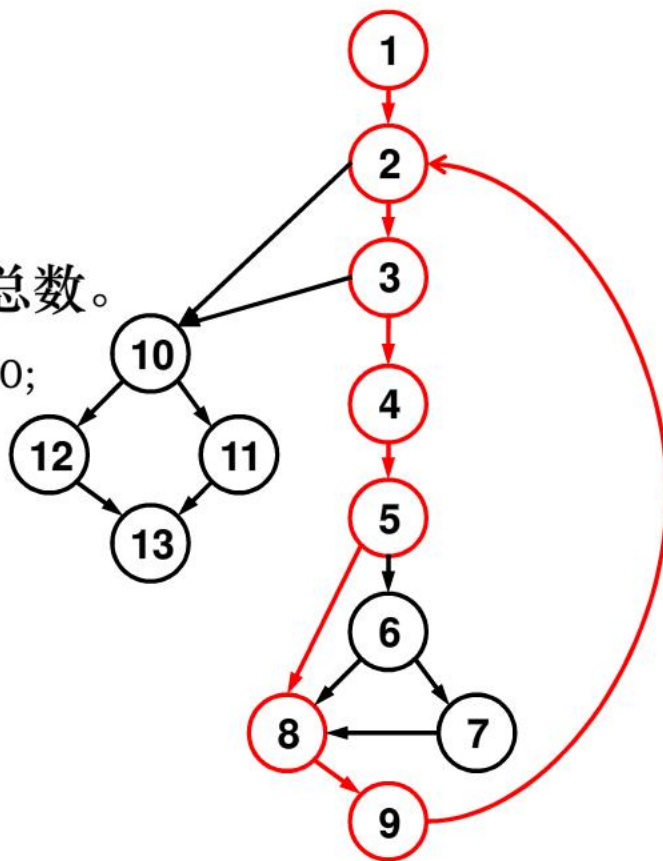
...

8: ENDIF

increment i by 1;

9: ENDDO

...



路径5的测试用例:

- 路径5: 1-2-3-4-5-6-8-9-2-...
- $value[i]$ =有效输入值, 其中 $i < 100$,
- $value[k] > maximum$, 其中 $k < i$
- 预期结果: 基于 k 的正确平均值和总数。

1: $i=1$; $total.input=total.valid=0$; $sum=0$;

2: DO WHILE $value[i] < -999$ T

3: AND $total.input < 100$ T

4: increment $total.input$ by 1;

5: IF $value[i] \geq minimum$ T

6: AND $value[i] \leq maximum$ F

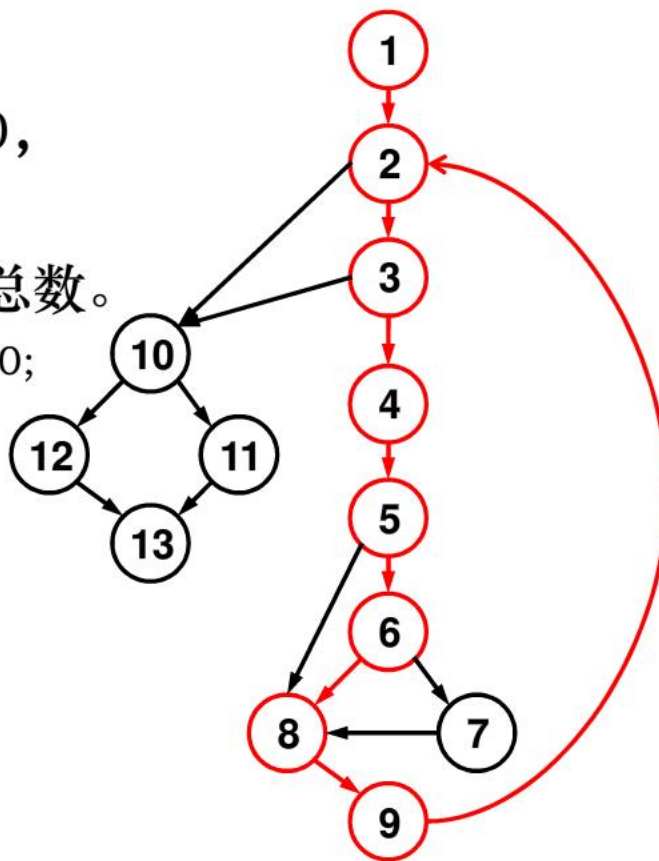
...

8: ENDIF

increment i by 1;

9: ENDDO

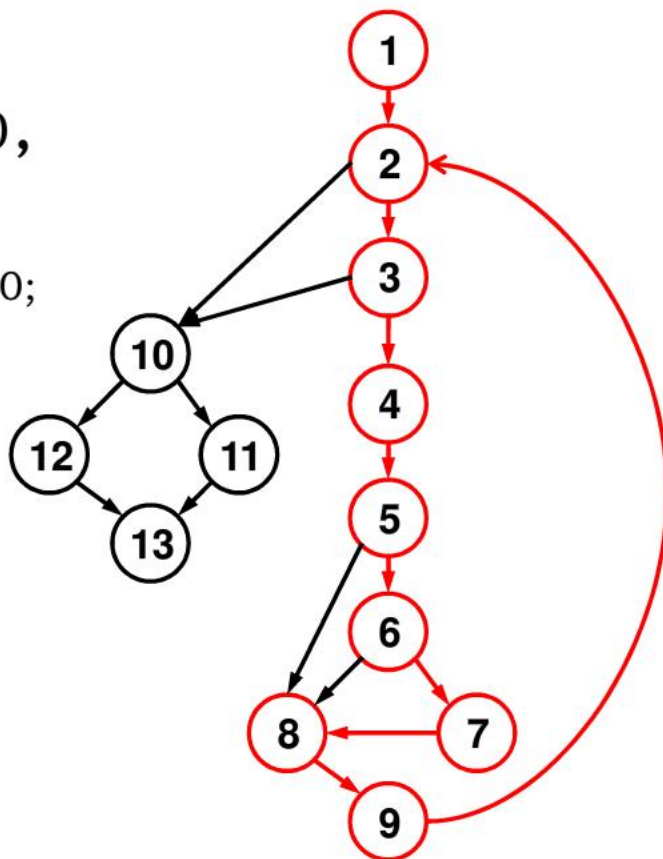
...



路径6的测试用例:

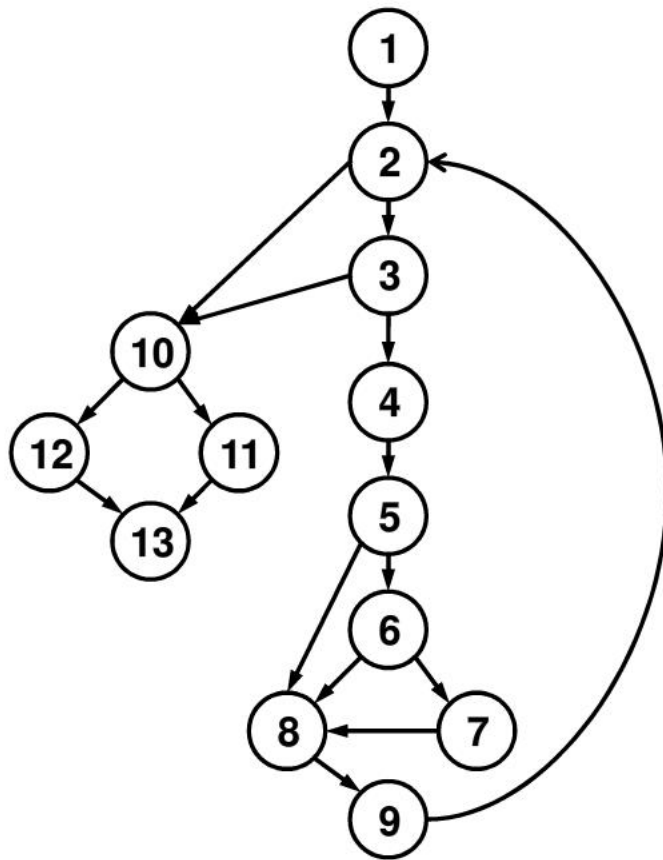
- 路径6: 1-2-3-4-5-6-7-8-9-2-...
- $\text{value}[i]$ =有效输入值, 其中 $i < 100$,
- 预期结果: 正确的平均值和总数。

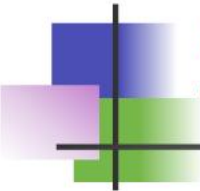
```
1:  i=1; total.input=total.valid=0; sum=0;
2:  DO WHILE value[i]<> -999           T
3:      AND total.input<100           T
4:      increment total.input by 1;
5:      IF value[i]>=minimum           T
6:          AND value[i]<=maximum     T
7:      THEN increment total.valid by 1;
          sum=sum+value[i];
8:      ENDIF
          increment i by 1;
9:  ENDDO
```



注意：

- 某些独立路径**不能以独立的方式测试**，就是说，程序的正常流程不能形成独立执行该路径所需要的数据组合。
- 这时，这些路径必须作为另一个路径的一部分来测试。





2. 条件测试

- 基本路径测试改进的必要性：
 - 尽管基本路径测试技术简单而且高效，但是仅有这种技术还不够，
 - 还需要使用其他控制结构测试技术，才能进一步提高白盒测试的质量。
- 条件测试技术的优点：
 - 设计出的测试用例，能够检查程序模块中包含的逻辑条件。

关系表达式

简单条件：

- 是一个布尔变量或一个关系表达式，
- 在布尔变量或关系表达式之前还可能有一个NOT算符。

关系表达式的形式：

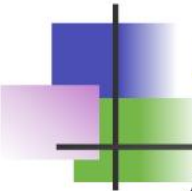
- $E1 < \text{关系运算符} > E2$

算术表达式

$<、\leq、=、\neq、>$ 或 $\geq$ 之一

复合条件：

- 由两个或多个简单条件、布尔算符和括弧组成。
- 布尔算符：OR (“|”) ， AND (“&”) 和 NOT。
- 布尔表达式：不包含关系表达式的条件称为布尔表达式。



条件成分的类型

■ 包括：

- 布尔算符、布尔变量、布尔括弧（括住简单条件或复合条件）、关系算符及算术表达式。

如果条件不正确，则至少条件的一个成分不正确。

■ 条件错误的类型：

- 布尔算符错（布尔算符不正确，遗漏布尔算符或有多余的布尔算符）
- 布尔变量错
- 布尔括弧错
- 关系算符错
- 算术表达式错



条件测试的测试重点及优点

■ 测试重点：

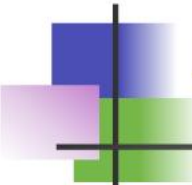
- 程序中的每个条件。

■ 优点：

- ①容易度量条件的测试覆盖率；
- ②程序内条件的测试覆盖率可指导附加测试的设计。

■ 目的

- 不仅检测程序条件中的错误，而且检测程序中的其他错误。



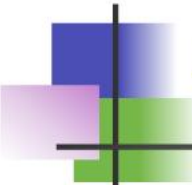
条件测试的目的

- 不仅检测程序条件中的错误，而且检测程序中的其他错误。
- 如果程序P的测试集能有效地检测P中条件的错误，则它很可能也可以有效地检测P中的其他错误。
- 此外，如果一个测试策略对检测条件错误是有效的，则很可能该策略对检测程序的其他错误也是有效的。



条件测试策略

- 分支测试
- 域测试
- 布尔变量测试
- BRO(branch and relational operator)测试



条件测试策略——分支测试

■特点：

- 可能是最简单的条件测试策略；

■要求：

- 对于复合条件C来说，C的真分支和假分支以及C中的每个简单条件，都应该至少执行一次。



条件测试策略——域测试

- 要求：对一个关系表达式执行3个或4个测试。

- 例如，对于形式为：

- $E1 < \text{关系算符} > E2$

的关系表达式的测试：

- 需要3个测试： $E1 > E2$ ， $E1 = E2$ ， $E1 < E2$ 的值。

- 发现关系算符的错误：

- 如果 $< \text{关系算符} >$ 错误，而 $E1$ 和 $E2$ 正确，则这3个测试能够测出。

- 发现 $E1$ 和 $E2$ 中的错误：

- $|E1 - E2|$ 尽可能小。

- $E1 > E2$ 或 $E1 < E2$ 的测试数据应该使这两个值之间的差别尽可能小。



条件测试策略——布尔变量测试

■ 测试次数：

- 包含 n 个变量的布尔表达式需要 2^n 个（每个变量分别取真或假这两个可能值的组合数）测试。

■ 作用及适用范围：

- 可以发现布尔算符、变量和括弧的错误。
- 仅在 n 很小时才是实用的。



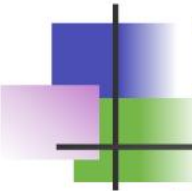
条件测试策略——BRO测试

产生：

- 在上述种种条件测试技术的基础上，K.C.Tai提出了一种被称为BRO(branch and relational operator)测试的条件测试策略。

■特点：

- 如果在条件中所有布尔变量和关系算符都只出现一次而且没有公共变量，
- 则BRO测试保证能发现该条件中的分支错和关系算符错。



BRO测试方法

- 利用条件C的条件约束来设计测试用例。
- 包含n个简单条件的条件C的条件约束定义为：
 $(D_1, D_2, \dots, D_n)$ ，其中：
 - $D_i (0 < i \leq n)$ 表示条件C中第i个简单条件的输出约束。
 - 如果在条件C的一次执行过程中，C中每个简单条件的输出都满足D中对应的约束，
 - 则称C的这次执行覆盖了C的条件约束D。
- 布尔变量B的输出约束：
 - B必须是真 (t) 或假 (f)。
- 关系表达式的输出约束：
 - 用符号 $>$ 、 $=$ 和 $<$ 指定。

例1 (BRO测试方法)

条件:

布尔变量

$$C_1: B_1 \ \& \ B_2$$

- C_1 的条件约束形式为: (D_1, D_2) , 其中:
 - D_1 : t、f;
 - D_2 : t、f。
- 例如, 值(t,f)是 C_1 的一个条件约束, 并由使 B_1 值为真 B_2 值为假的测试所覆盖。
- C_1 的约束集: $\{(t,t),(f,t),(t,f)\}$ 。 由 B_1 、 B_2 的取值确定。
- BRO测试策略:
 - 覆盖 C_1 条件约束集的测试, 可发现 C_1 中布尔算符的错误。
 - 即如果 C_1 因布尔算符错误而不正确, 则至少上述约束集中的一个约束将迫使 C_1 失败。

例2 (BRO测试方法)

■ 条件:

布尔变量

算术表达式

$$C_2: B_1 \ \& \ (E_3=E_4)$$

■ C_2 的条件约束形式为: (D_1, D_2) , 其中:

■ D_1 : t、f;

■ D_2 : >、=、<。 为关系表达式。

■ C_2 的约束集 (修改 C_1 的约束集得到):

■ $\{(t,=), (f,=), (t,<), (t,>)\}$ 。(其中: =为t, <或>为: f)

■ BRO测试策略:

■ 覆盖 C_2 条件约束集的测试, 保证可发现 C_2 中布尔算符和关系运算符的错误。

例3 (BRO测试方法)

■ 条件:

算术表达式

$$C_3: (E_1=E_2) \ \& \ (E_3=E_4)$$

■ C_3 的条件约束形式为: (D_1, D_2) , 其中:

■ $D_1: >、=、<;$
■ $D_2: >、=、<。$ } 关系表达式

■ C_3 的约束集 (修改 C_2 的约束集得到):

■ $\{(>,=), (=,=), (<,=), (>,<), (>,>)\}$ 。

■ 其中: =为t, <或>为f

■ BRO测试策略:

■ 覆盖 C_3 条件约束集的测试, 保证可发现 C_3 中关系运算符的错误。



3. 循环测试

■ 问题提出：

- 循环是绝大多数软件算法的基础，但是，在测试软件时却往往未对循环结构进行足够的测试。

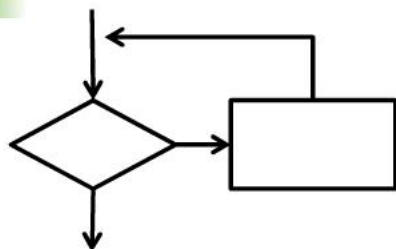
■ 循环测试：

- 是一种白盒测试技术，它专注于测试循环结构的有效性。

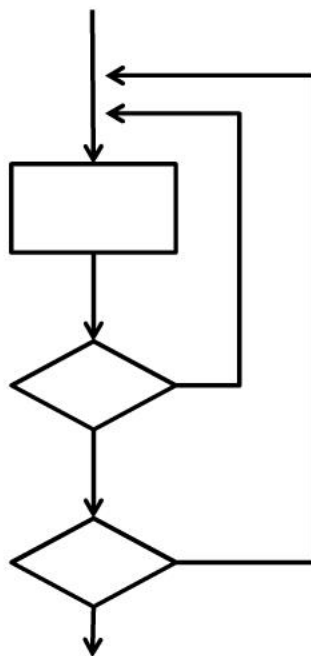
■ 结构化的程序中的3种循环：

- 简单循环、串接循环、嵌套循环。如图所示。
- 下面分别讨论这3种循环的测试方法。

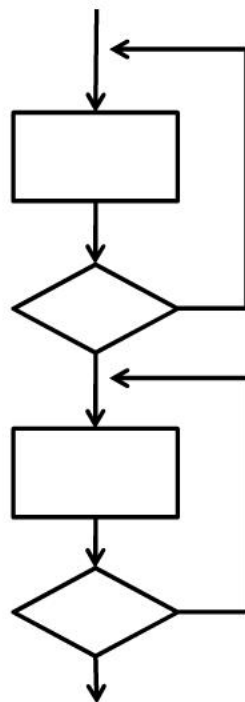
结构化的程序中的3种循环



简单循环



嵌套循环



串接循环

图 3种循环

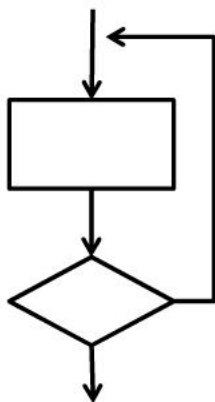
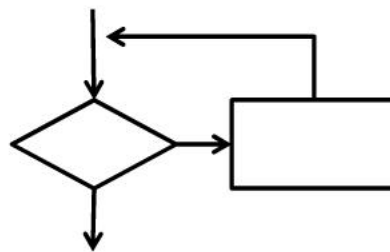
3. 循环测试——简单循环

■ 测试策略：

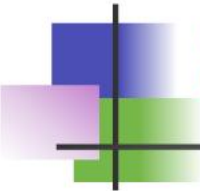
- 使用下列测试集测试简单循环，
- 其中 n 是允许通过循环的最大次数。

■ 测试集(5种)

- 跳过循环。
- 只通过循环一次。
- 通过循环两次。
- 通过循环 m 次，其中 $m < n - 1$ 。
- 通过循环 $n - 1, n, n + 1$ 次。



简单循环



3. 循环测试——嵌套循环

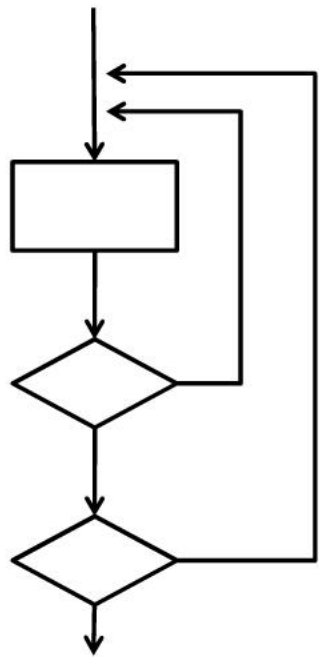
■问题：

- 如果把简单循环的测试方法直接应用到嵌套循环，
 - 则测试数就会随嵌套层数的增加按几何级数增长，
 - 导致不切实际的测试数目。
- B.Beizer提出了一种减少测试数的方法(简)：
- 从最内层循环开始。
 - 最内层循环使用简单循环测试方法。
 - 由内向外，对下一个循环进行测试。
 - 继续进行下去，直到测试完所有循环。

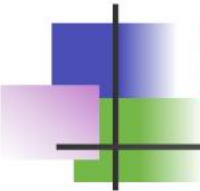
3. 循环测试——嵌套循环(续)

■ B.Beizer的减少测试数的方法：

- 从最内层循环开始测试，把所有其他循环都设置为最小值。
- 对最内层循环使用简单循环测试方法，而使外层循环的迭代参数（例如，循环计数器）取最小值，并为越界值或非法值增加一些额外的测试。
- 由内向外，对下一个循环进行测试，但保持所有其他外层循环为最小值，其他嵌套循环为“典型”值。
- 继续进行下去，直到测试完所有循环。



嵌套循环



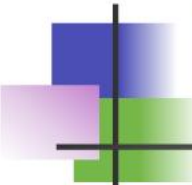
3. 循环测试——串接循环

- 循环都彼此独立：

- 如果串接循环的各个循环都彼此独立，
- 则使用测试简单循环的方法来测试串接循环。

- 两个循环不独立：

- 如果两个循环串接，而且第一个循环的循环计数器值是第二个循环的初始值，则这两个循环并不是独立的。
- 当循环不独立时，则使用测试嵌套循环的方法来测试串接循环。



5.8 黑盒测试技术

- 黑盒测试重点：
 - 测试软件功能。
- 黑盒测试的作用：
 - 不能取代白盒测试，它是与白盒测试互补的测试方法，它很可能发现白盒测试不易发现的其他类型的错误。
- 黑盒测试力图发现的错误类型(5种):
 - ①功能不正确或遗漏了功能；
 - ②界面错误；
 - ③数据结构错误或外部数据库访问错误；
 - ④性能错误；
 - ⑤初始化和终止错误。